Collisioni, stima

- Lunghezza media delle catene di overflow, al variare di
  - Numero di record esistenti: T
  - Numero di blocchi: B
  - Fattore di blocco: F
  - Coefficiente di riempimento: T/(FxB)

	1	2	3	5	10	(F)
.5	0.5	0.177	0.087	0.031	0.005	
.6	0.75	0.293	0.158	0.066	0.015	
.7	1.167	0.494	0.286	0.136	0.042	
.8	2.0	0.903	0.554	0.289	0.110	
.9	4.495	2.146	1.377	0.777	0.345	
T/(FxB)						

Costo, stima

- Costo (uno più la lunghezza media), nelle stesse condizioni
  - Numero di record esistenti: T
  - Numero di blocchi: B
  - Fattore di blocco: F
  - Coefficiente di riempimento: T/(FxB)

	1	2	3	5	10	(F)
.5	1.5	1.177	1.087	1.031	1.005	
.6	1.75	1.293	1.158	1.066	1.015	
.7	2.167	1.494	1.286	1.136	1.042	
.8	3.0	1.903	1.554	1.289	1.110	
.9	5.495	3.146	2.377	1.777	1.345	
T/(FxB)						

File hash, osservazioni

- Costo medio di poco superiore all'unità
    - il caso peggiore è molto costoso ma talmente improbabile da poter essere ignorato
  - è l'organizzazione più efficiente per l'accesso diretto basato su valori della chiave con condizioni di uguaglianza (accesso puntuale)
- Non è efficiente per ricerche basate su intervalli
- I file hash "degenerano" se si riduce lo spazio sovrabbondante: **funzionano solo con file la cui dimensione non varia molto nel tempo**
  - Esistono però tecniche per superare questo difetto:
    - Hashing dinamico (estendibile, lineare)

Hashing estendibile

- Il numero di blocchi (o bucket) non è predefinito
- Utilizza una directory dei blocchi (anch'essa di dimensione variabile, anche se molto lentamente)
- Fa riferimento alla rappresentazione binaria del valore di hash

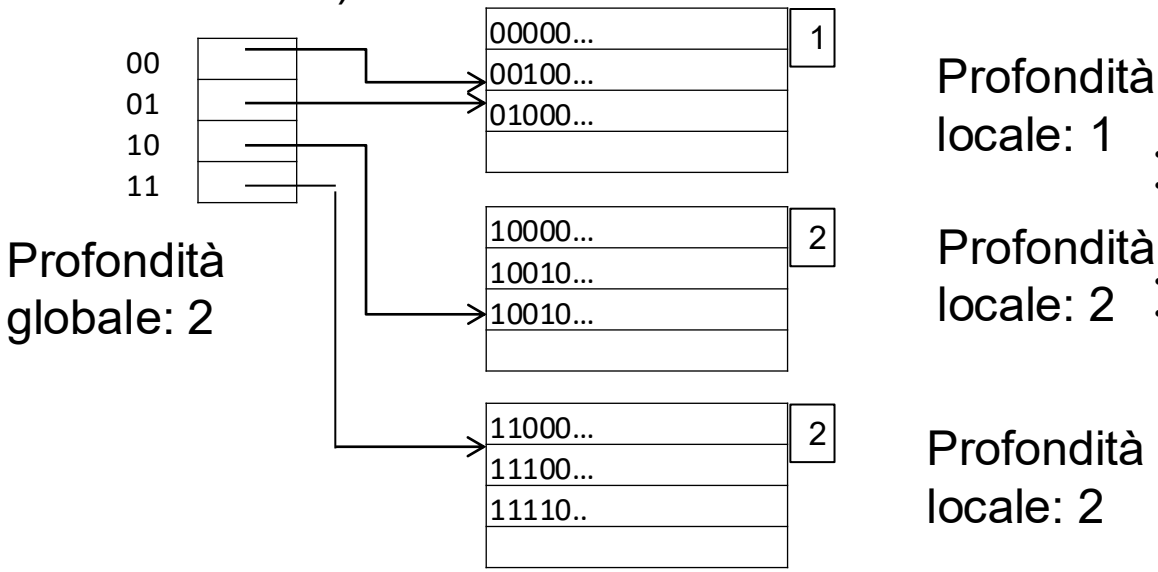
Hashing estensibile

- Funzione hash di **k bit** (k anche molto grande)
- Directory con 2^d elementi, con d<=k
- Blocchi utilizzati in numero in generale inferiore a 2^d,
  - più elementi della directory possono puntare allo stesso blocco
- Se un blocco si satura, viene diviso in due
- Se si deve dividere un blocco cui punta un solo elemento della directory, allora si raddoppia la directory

  (vedere insieme al lucido seguente)

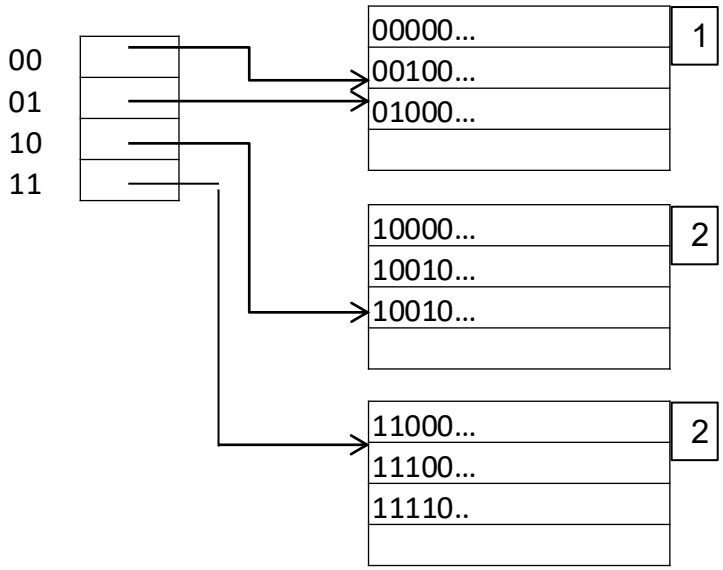
Esempio

- **Profondità globale**: numero di bit usati per la directory, d=2
- Directory con 2^d =4 elementi, massimo 4 blocchi, ma sono meno, se non ci sono record che li utilizzano
- **Profondità locale** di un blocco: numero di bit iniziali "comuni" ai record nel blocco, presenti o potenziali, sulla base dei riferimenti nella directory, (uguale alla profondità globale se al blocco punta un solo elemento)



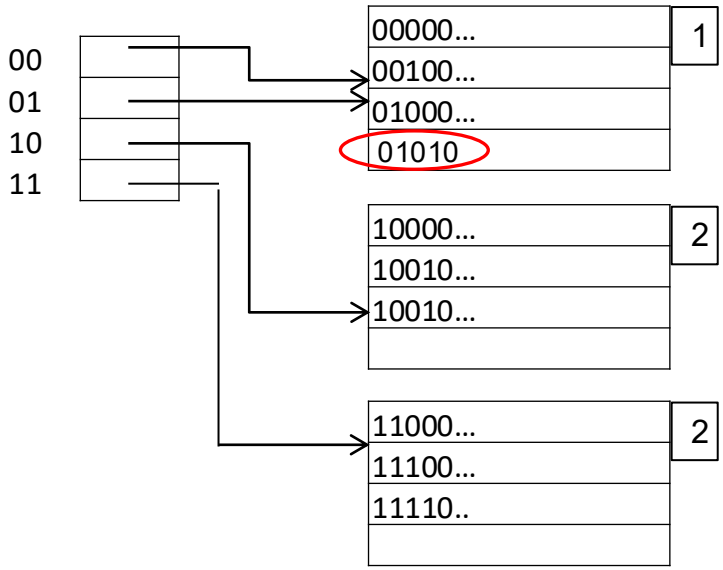
- Directory con 2^d elementi, con d<=k
- Blocchi utilizzati in numero in generale inferiore a 2^d,
  - più voci della directory possono puntare allo stesso blocco
- Se un blocco si satura, viene diviso in due
- Se si deve dividere un blocco cui punta un solo elemento, allora si raddoppia la directory

Se un blocco si satura …



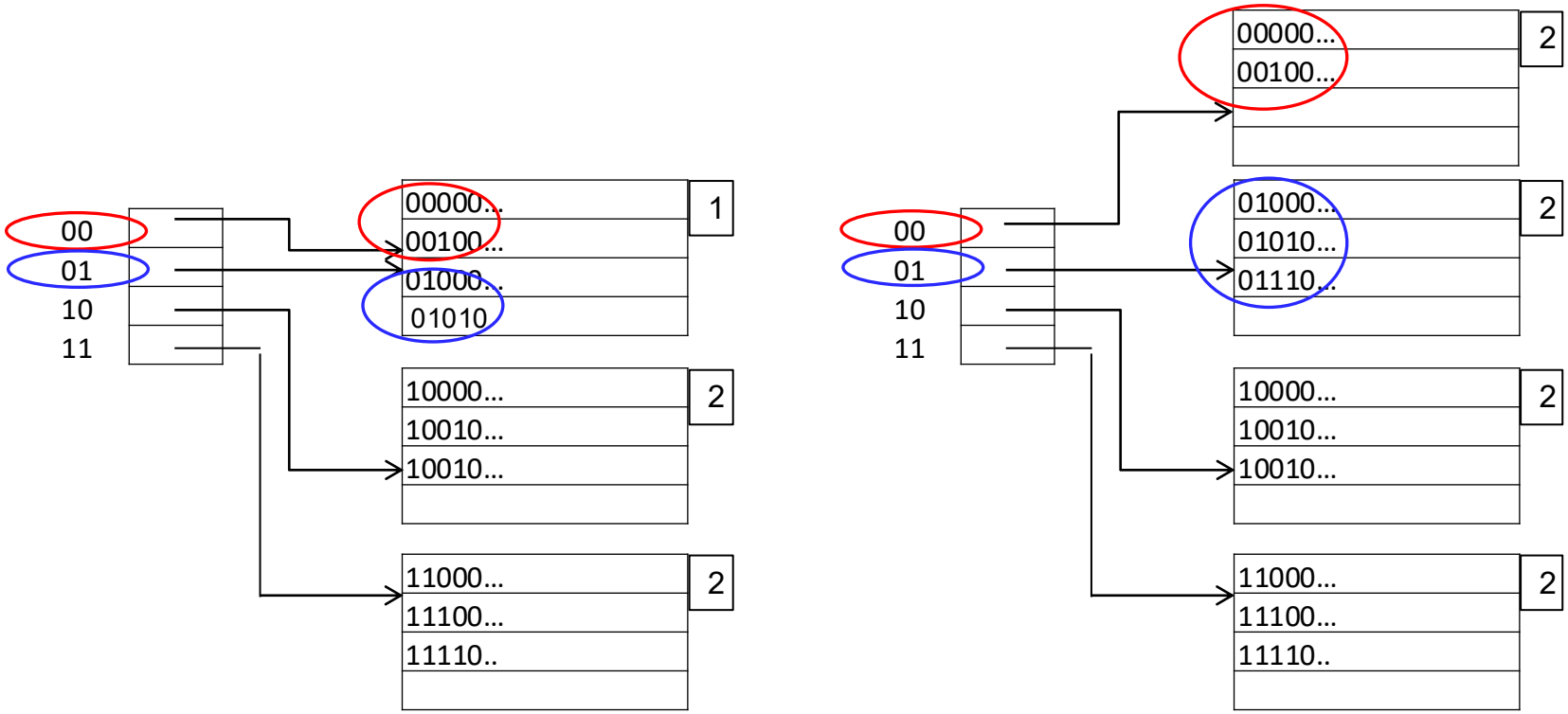
- Inseriamo due record 01010… e 01110…
- Il primo blocco si satura e viene diviso

Se un blocco si satura …



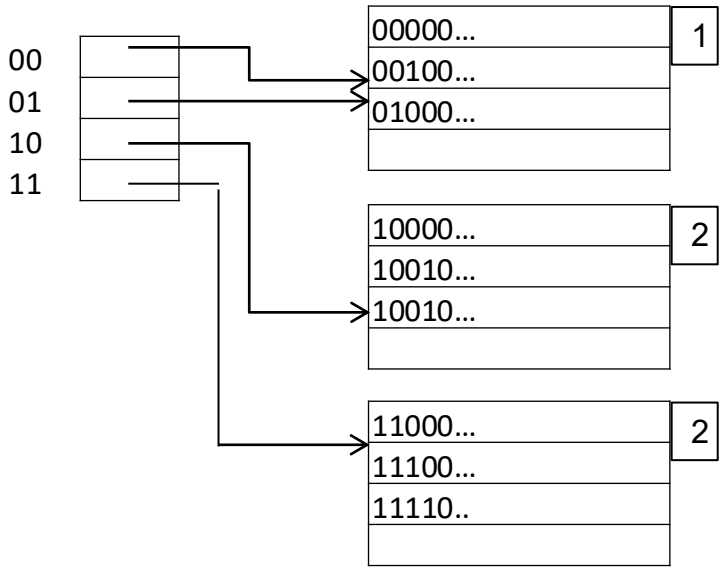
- Inseriamo 01010

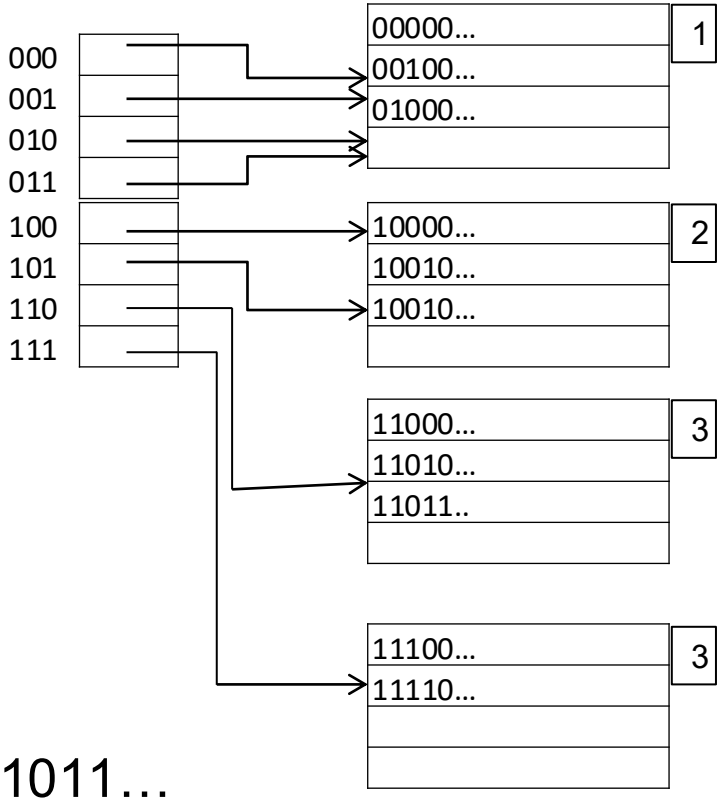
Se un blocco si satura …



- Inseriamo 01110
- Andrebbe nel primo blocco, ma non c'è posto

Se si satura un blocco con profondità locale pari alla profondità globale …





- Inseriamo due record 11010… e 11011…
- L'ultimo blocco va diviso e va raddoppiata la directory

Hashing estensibile, commenti

- Costo aggiuntivo, l'accesso alla directory
  - Costo due, directory più blocco (più le riorganizzazioni, molto rare, quindi possiamo assumere due come riferimento, la media sarà di poco superiore a due)
  - Però se si usa di frequente, la directory può essere nei buffer e quindi il costo si avvicina all'unità
- Supera le limitazioni dell'hashing statico in caso di dimensione variabile nel tempo

Strutture primarie e secondarie

- Ricordiamo
  - strutture **primarie**: determinano collocazione/posizionamento dei record
  - strutture **secondarie**: elementi aggiuntivi per l'accesso efficiente
- Le strutture sequenziali (disordinate, ordinate, etc.) sono strutture **primarie**
- Le strutture hash, come viste finora, sono pure **primarie** (se ne può avere una variante secondaria)
- Vediamo ora strutture che nascono come **secondarie** (anche se loro varianti possono essere primarie)
- Cominciamo con alcuni esempi

Una biblioteca e il suo catalogo

Indice analitico

Abort, 190
Access, *419–440*
    Avvio di, 420
    Cammini di join in, 426
    Definizione di tabelle in, 421
    Domini in, 421–425
    Interprete SQL in, 435
    Macro in, 438
    Maschere in, 436
    Popolamento delle tabelle in, 428
    QBE (query by example) in, 429–435
    Report in, 438
Accesso, Controllo di, 187
Accorpamento
    di associazioni, 320, 330–331
    di entità, 320, 330
Acidità delle transazioni, 191
ADO, ActiveX Data Object, 213
ANSI, 111
`any`, 155
Applicazioni
    in DB2, 451–455
`asc`, 142
Asserzioni in SQL, 176
Associazione
    verifica di normalizzazione, 409–410
Associazioni
    Accorpamento di, 320, 330–331
    in UML, 256, 258–260
    nel modello E-R, 236–238
    Partizionamento di, 320, 330–331
Atomicità, 191
Attributi
    Cardinalità di, 243–244
    composti, 240
    di relazione, 37
    Eliminazione di — multivalore, 329
    in UML, 257
    nel modello E-R, 238–240
Autorizzazioni, 19, 187
`avg`, 144

Indice

Prefazione **ix**

1 Introduzione **15**
1.1 Sistemi informativi, informazioni e dati 15
1.2 Basi di dati e sistemi di gestione di basi di dati 17
1.3 Modelli dei dati 20
1.4 Linguaggi e utenti delle basi di dati 24
1.5 Vantaggi e svantaggi dei DBMS 27

Parte Prima Basi di dati relazionali: modello e linguaggi **31**

2 Il modello relazionale **33**
2.1 Il modello relazionale: strutture 33
2.2 Vincoli di integrità 47
2.3 Conclusioni 57

Indici di file

- **Indice**:
  - struttura ausiliaria per l'accesso (efficiente) ai record di un file sulla base dei valori di un campo (o di una "concatenazione di campi") detto **chiave** (o, meglio, **pseudochiave**, perché non è necessariamente identificante);
- Idea fondamentale: l'indice analitico di un libro: lista di coppie (termine,pagina), ordinata alfabeticamente sui termini, posta in fondo al libro e separabile da esso
- Un indice ***I*** di un file ***F*** è un altro file, con record a due campi: chiave e indirizzo (dei record di ***F*** o dei relativi blocchi), ordinato secondo i valori della chiave

Tipi di indice

- indice **primario**:
  - su un campo sul cui ordinamento è basata la memorizzazione (detti anche indici di **cluster**, anche se talvolta si chiamano **primari** quelli su una chiave identificante e di **cluster** quelli su una pseudochiave non identificante)
  - Attenzione:
    - In alcuni sistemi viene chiamato "indice primario" un indice (di solito secondario) definito sulla chiave primaria. Noi **non** utilizziamo questa terminologia
- indice **secondario**
  - su un campo con ordinamento diverso da quello di memorizzazione

Indice analitico

Abort, 190
Access, *419–440*
    Avvio di, 420
    Cammini di join in, 426
    Definizione di tabelle in, 421
    Domini in, 421–425
    Interprete SQL in, 435
    Macro in, 438
    Maschere in, 436
    Popolamento delle tabelle in, 428
    QBE (query by example) in, 429–435
    Report in, 438
Accesso, Controllo di, 187
Accorpamento
    di associazioni, 320, 330–331
    di entità, 320, 330
Acidità delle transazioni, 191
ADO, ActiveX Data Object, 213
ANSI, 111
`any`, 155
Applicazioni
    in DB2, 451–455
`asc`, 142
Asserzioni in SQL, 176
Associazione
    verifica di normalizzazione, 409–410
Associazioni
    Accorpamento di, 320, 330–331
    in UML, 256, 258–260
    nel modello E-R, 236–238
    Partizionamento di, 320, 330–331
Atomicità, 191
Attributi
    Cardinalità di, 243–244
    composti, 240
    di relazione, 37
    Eliminazione di — multivalore, 329
    in UML, 257
    nel modello E-R, 238–240
Autorizzazioni, 19, 187
`avg`, 144

Indice

Prefazione **ix**

1 Introduzione **15**
1.1 Sistemi informativi, informazioni e dati 15
1.2 Basi di dati e sistemi di gestione di basi di dati 17
1.3 Modelli dei dati 20
1.4 Linguaggi e utenti delle basi di dati 24
1.5 Vantaggi e svantaggi dei DBMS 27

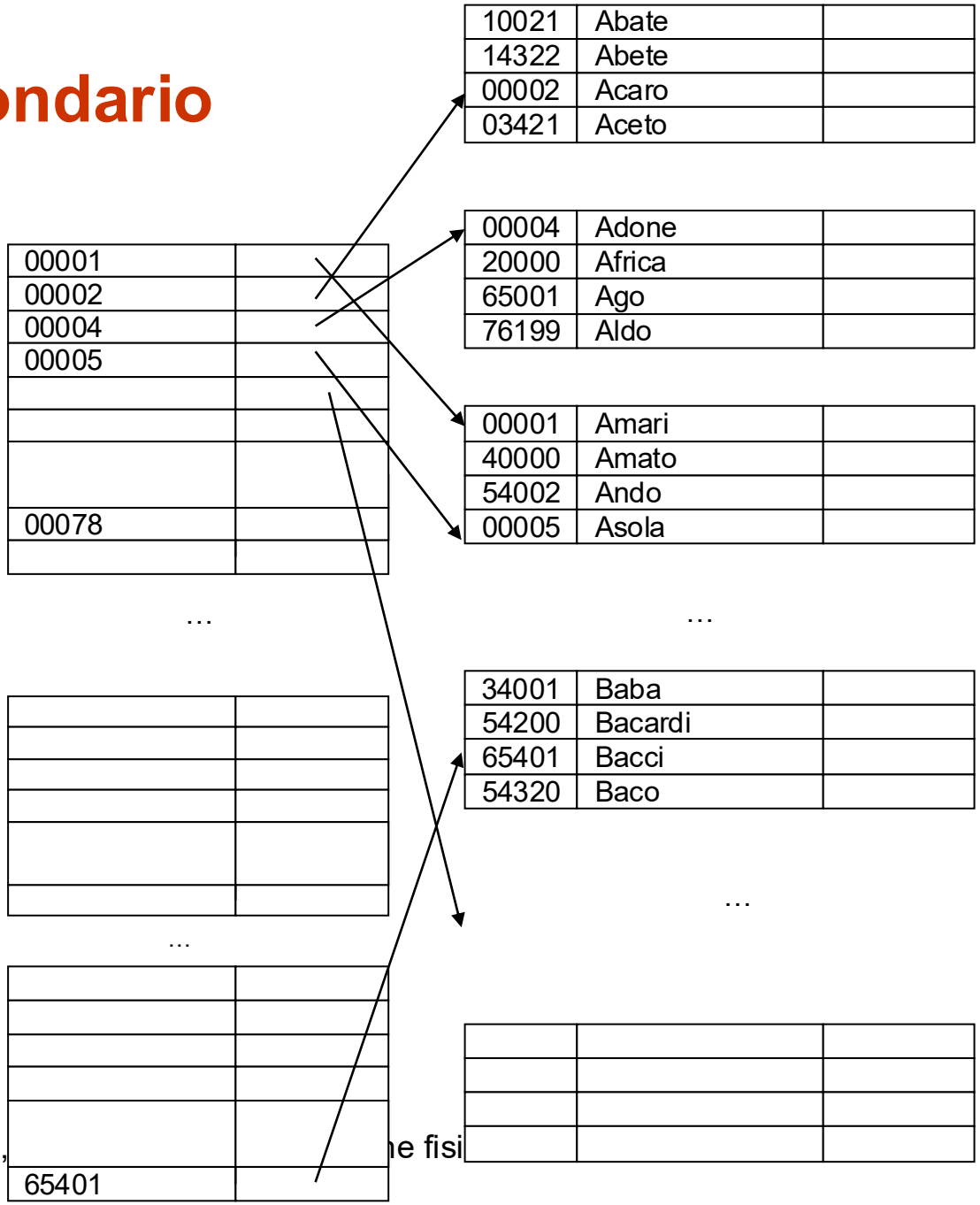
Parte Prima Basi di dati relazionali: modello e linguaggi **31**

2 Il modello relazionale **33**
2.1 Il modello relazionale: strutture 33
2.2 Vincoli di integrità 47
2.3 Conclusioni 57

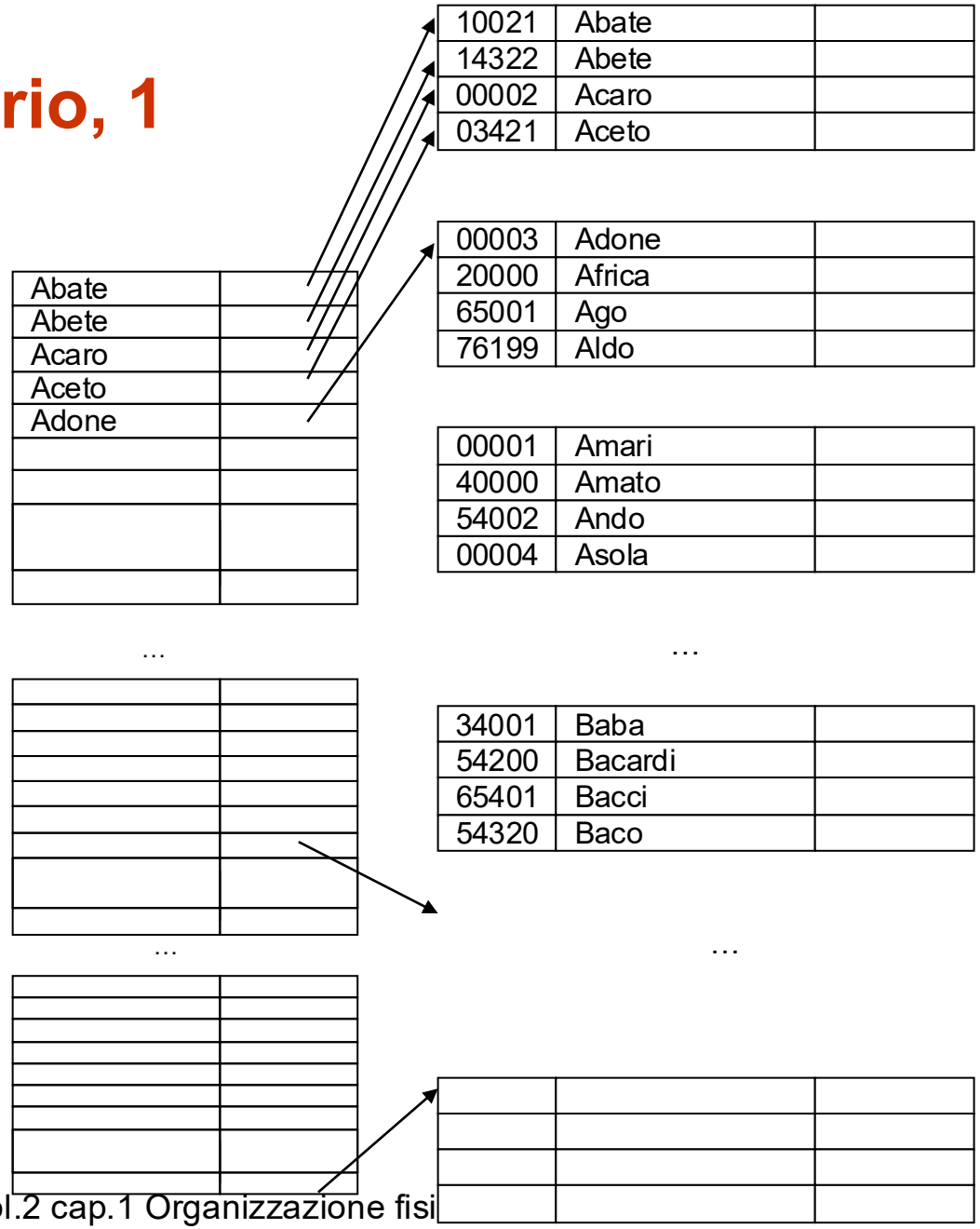
Tipi di indice, commenti

- Nell'esempio del libro
  - indice generale: primario (pagine in ordine)
  - indice analitico: secondario (pagine in disordine)
- Ogni file può avere al più un indice primario (salvo il caso di un indice su un "prefisso" dell'altro, ad esempio uno su cognome e l'altro su cognome e nome) e un numero qualunque di indici secondari (su campi diversi). Esempio:
  - In alcune guide turistiche abbiamo due indici "secondari"
    - indice dei luoghi
    - indice degli artisti
  - Le biblioteche hanno (o avevano) cataloghi
    - per titolo
    - per autore
    - per argomento
- Un file hash non può avere un indice primario

Indice secondario

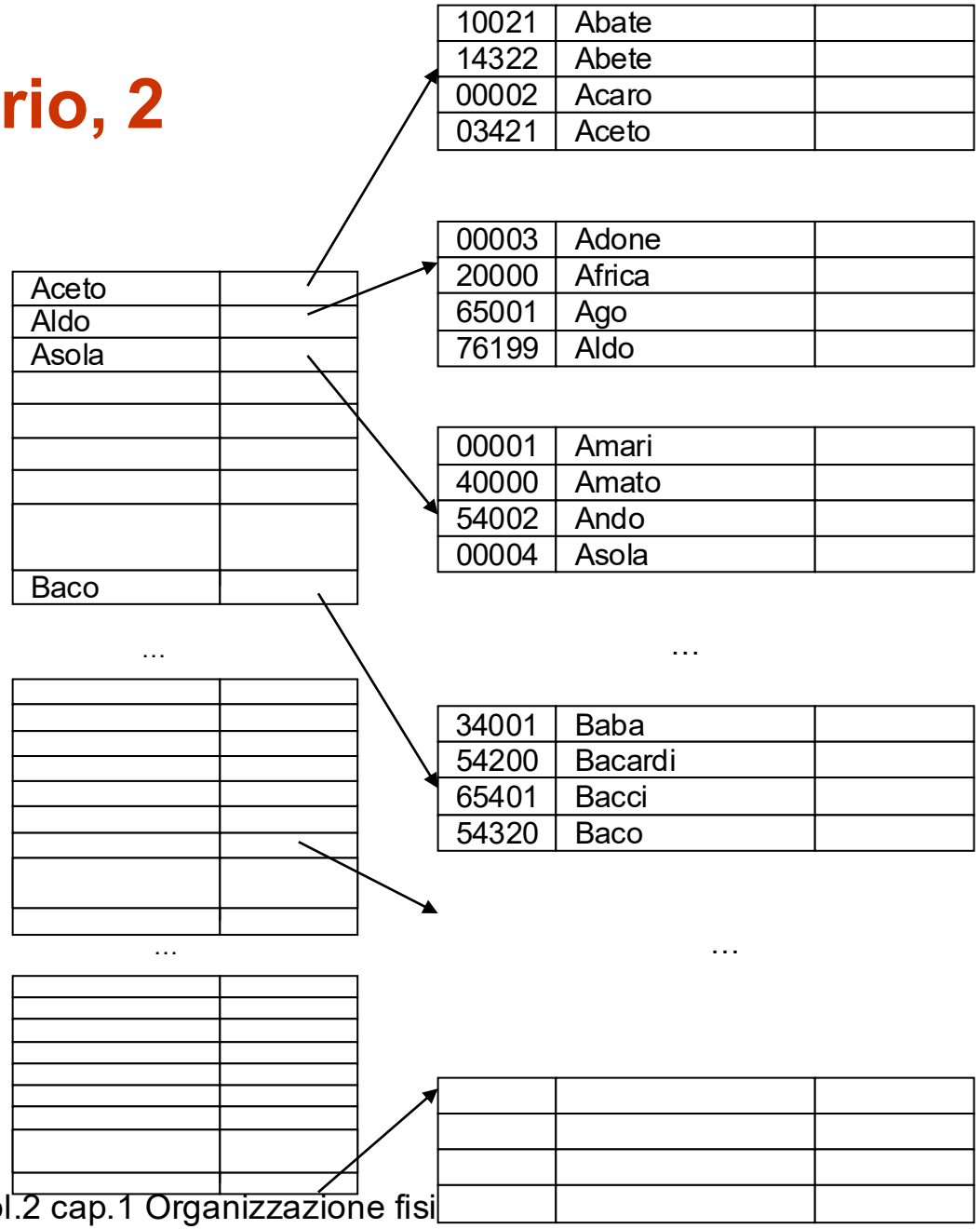


Indice primario, 1



Servono tutti i riferimenti?

Indice primario, 2



Tipi di indice, ancora

- indice **denso**:
  - contiene tutti i valori della chiave (e quindi, per indici su campi identificanti, un riferimento per ciascun record del file)
- indice **sparso**:
  - contiene solo alcuni valori della chiave e quindi (anche per indici su campi identificanti) un numero di riferimenti inferiore rispetto ai record del file

- Un indice secondario
  - deve essere denso
- Un indice primario
  - può essere sparso (e di solito lo è)
  - può essere denso
    - per verificare se un record è presente o meno
    - per permettere operazioni sugli indirizzi

Indici densi, un’osservazione

- Si possono usare, come detto, puntatori ai blocchi oppure puntatori ai record
  - I puntatori ai blocchi sono più compatti
  - I puntatori ai record permettono di
    - semplificare alcune operazioni (effettuate solo sull'indice, senza accedere al file se non quando indispensabile)

Indici su campi non chiave

- Ci sono (in generale) **più record per un valore della (pseudo)chiave**
  - primario sparso, possibili semplificazioni:
    - puntatori solo a blocchi con valori “nuovi”
  - primario denso:
    - per ogni record, una coppia con valore della chiave e riferimento (quindi i valori della chiave si ripetono)
    - valore della chiave una sola volta, seguito dalla lista di riferimenti ai record con quel valore
    - valore della chiave, seguito dal riferimento al primo record con quel valore (perde i benefici dell’indice primario denso legati alla possibilità di lavorare sui puntatori)
  - secondario (denso):
    - una coppia con valore della chiave e riferimento per ogni record (quindi i valori della chiave si ripetono)
    - un livello (di “indirezione”) in più: per ogni valore della chiave l’indice contiene un record con riferimento al blocco di una struttura intermedia che contiene riferimenti ai record

Dimensioni dell'indice (per semplicità, su campo identificante)

- L numero di record nel file
- B dimensione dei blocchi
- R lunghezza dei record (fissa)
- K lunghezza del campo chiave
- P lunghezza degli indirizzi (ai blocchi)
- K+P lunghezza dei record degli indici

Fattore di blocco del file:	B/R
N. di blocchi per il file (circa):	$N_F = L / (B/R)$
Fattore di blocco di un indice:	$B/(K+P)$
N. di blocchi per un indice denso:	$N_D = L / (B/(K+P))$
N. di blocchi per un indice sparso:	$N_S = N_F / (B/(K+P))$

- L numero di record nel file 1.000.000
- B dimensione dei blocchi 4KB
- R lunghezza dei record (fissa per semplicità) 100B
- K lunghezza del campo chiave 4B
- P lunghezza degli indirizzi (ai blocchi) 4B
- K+P lunghezza dei record degli indici 8B

Fattore di blocco del file

B/R = 40

Blocchi del file

$N_F = L / (B/R) = 25.000$

Fattore di blocco degli indici

B/(K+P) = 500

Blocchi di indice denso

$L / (B/(K+P)) = 2.000$

Blocchi di indice sparso

$N_F / (B/(K+P)) = 50$

Caratteristiche degli indici

- Accesso diretto (sulla chiave) efficiente, sia puntuale sia per intervalli
  - È efficiente perché, anche nel modello semplice visto finora, si può lavorare sulll'indice, che è più piccolo del file ed è ordinato
- Scansione sequenziale ordinata efficiente:
  - Tutti gli indici (in particolare quelli secondari) forniscono un **ordinamento logico** sui record del file; con numero di accessi pari al numero di record del file (a parte qualche beneficio dovuto alla bufferizzazione)
- Modifiche della chiave, inserimenti, eliminazioni inefficienti (come nei file ordinati)
  - tecniche per alleviare i problemi:
    - file o blocchi di overflow
    - marcatura per le eliminazioni
    - riempimento parziale
    - blocchi collegati (non contigui)
    - riorganizzazioni periodiche
    - … (vedremo più avanti)

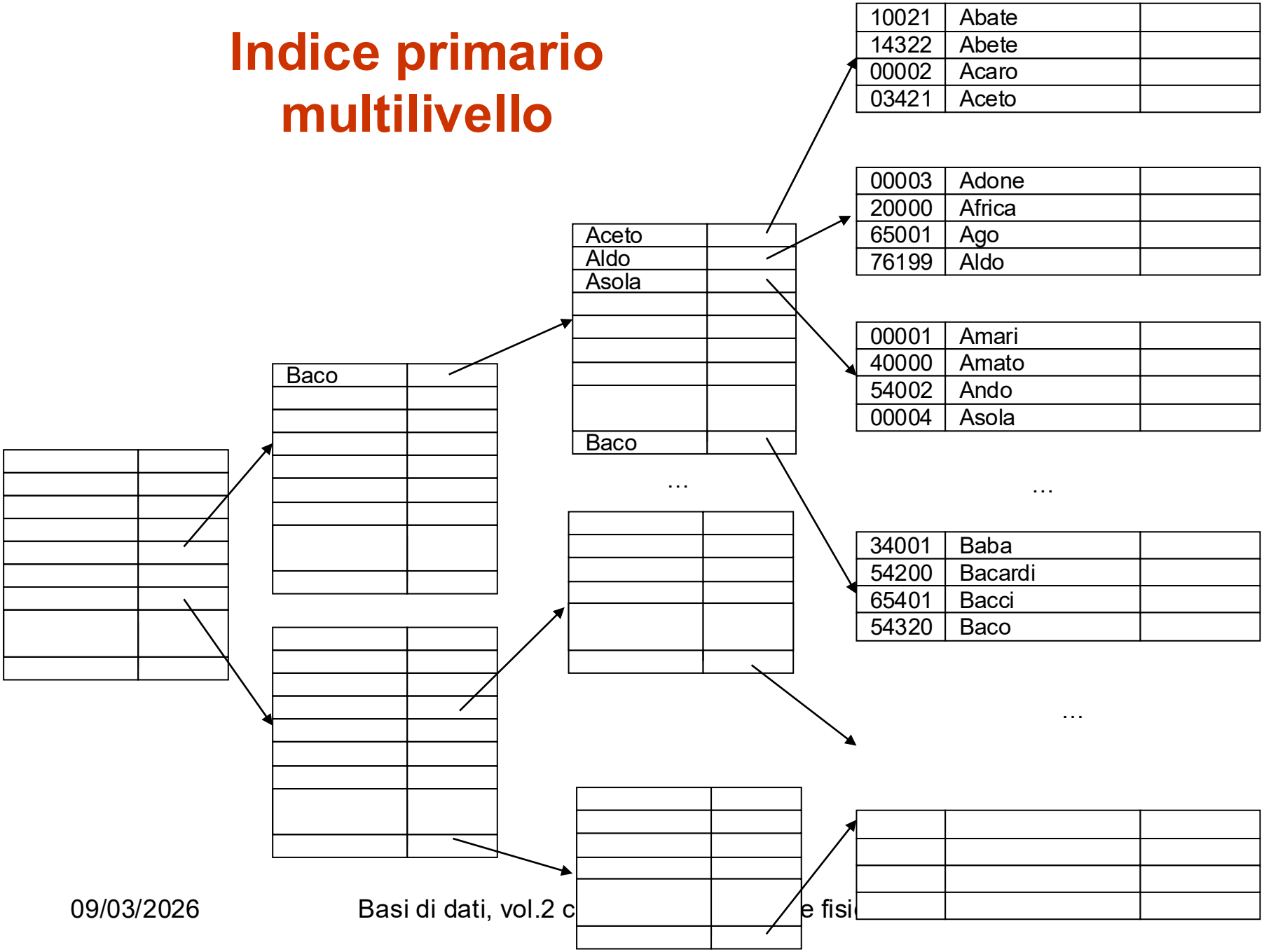
Limiti e possibili miglioramenti

- Gli indici visti finora
  - richiedono scansioni (o ricerche binarie, se la memorizzazione è contigua), il che può essere oneroso
  - sono onerosi per la rigidità delle strutture ordinate
- Gli indici reali superano queste limitazioni
  - organizzazione multilivello (per evitare scansioni)
  - struttura flessibile (grazie a ridondanza e riorganizzazioni locali)

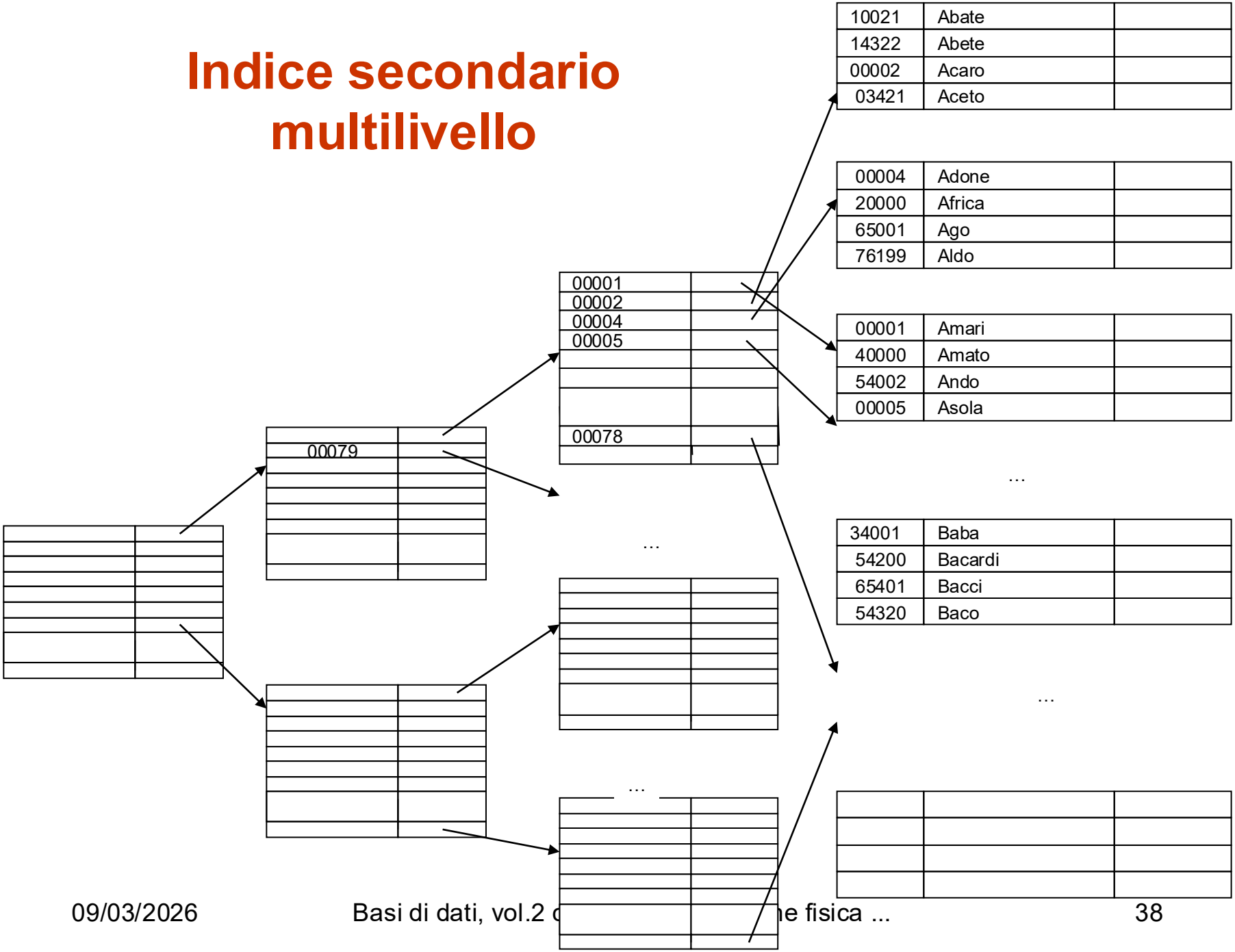
Indici multilivello

- Gli indici sono file essi stessi e quindi ha senso costruire indici sugli indici, per evitare di fare ricerche fra blocchi diversi (che potrebbero richiedere scansioni sequenziali)
- L'indice è ordinato e quindi l'indice sull'indice è primario (e sparso)
- Il tutto a più livelli, fino ad avere un livello con un solo blocco

Indice primario multilivello



Indice secondario multilivello



Indici multilivello

- I livelli sono di solito abbastanza pochi, perché
  - l'indice è ordinato, quindi l'indice sull'indice può essere sparso (e lo è sempre)
  - i record dell'indice sono piccoli
- N_j numero di blocchi al livello j dell'indice (circa):
  - $N_j = N_{j-1} / (B/(K+P))$
- Termine spesso usato:
  - Fan-out dell'indice: numero di blocchi cui un blocco fa riferimento, è pari al fattore di blocco dell'indice

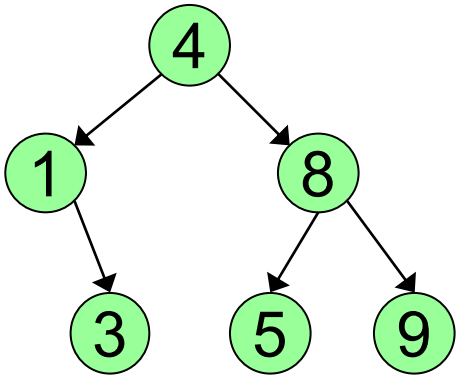
- Negli esempi numerici (B/(K+P) = 4.000/8=500)
  - Denso: $N_1 = 2.000$, $N_2 = 4$, $N_3 = 1$
  - Sparso: $N_1 = 50$, $N_2 = 1$

Indici, problemi

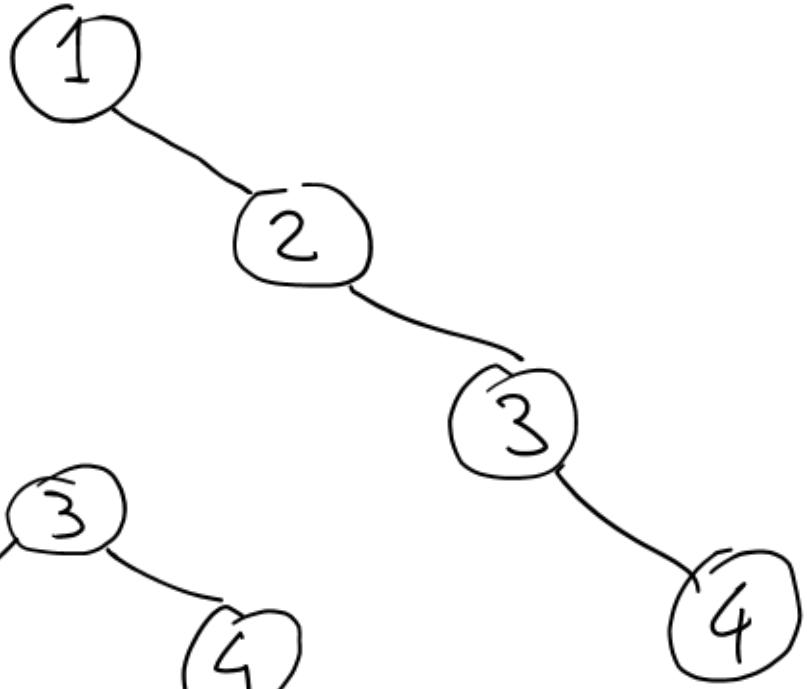
- Tutte le strutture di indice viste finora sono basate su **strutture ordinate e quindi sono poco flessibili** in presenza di elevata dinamicità
- Gli indici utilizzati dai DBMS sono più sofisticati:
  - indici dinamici multilivello: B-tree (intuitivamente: alberi di ricerca bilanciati)
    - Arriviamo ai B-tree per gradi
      - Alberi binari di ricerca
      - Alberi n-ari di ricerca
      - Alberi n-ari di ricerca bilanciati

Albero binario di ricerca

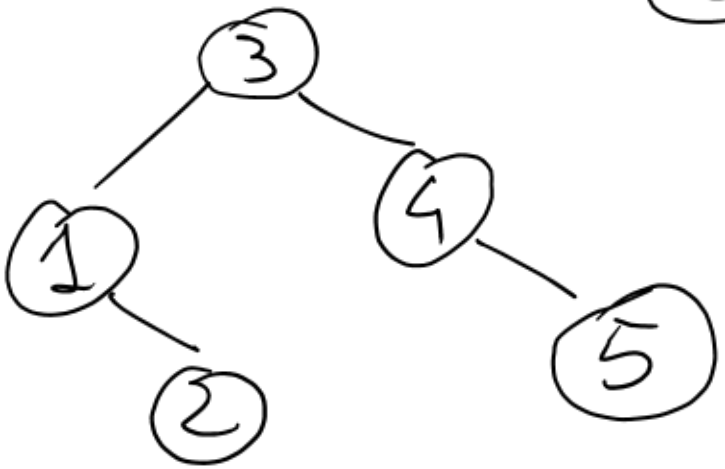
- Albero binario etichettato in cui per ogni nodo il sottoalbero sinistro contiene solo etichette minori di quella del nodo e il sottoalbero destro etichette maggiori
- tempo di ricerca (e inserimento), pari alla profondità:
  - logaritmico nel caso “medio” (assumendo un ordine di inserimento casuale)



1, 2, 3, 4



3, 4, 1, 5, 2



Un concetto visto nel corso di Algoritmi e strutture dati

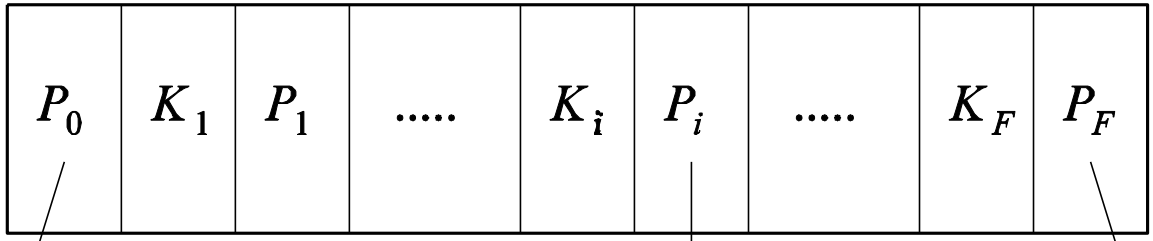
- Alberi rosso-neri
  - alberi binari bilanciati
  - operazioni di rotazione per mantenere il bilanciamento
- Non approfondiamo, ma vedremo qualcosa di simile fra poco

Albero di ricerca di ordine (o grado) P

- Ogni nodo ha
  - (fino a) P figli
  - (fino a) P-1 etichette, ordinate
- Nell'i-esimo sottoalbero ci sono
  - etichette maggiori della (i-1)-esima etichetta e
  - minori della i-esima

Albero di ricerca di ordine (grado) 3

Nodi in un albero di ricerca di ordine (grado) F+1



Ogni ricerca o modifica comporta la visita di un cammino radice foglia

sotto-albero che contiene le chiavi $K < K_1$

sotto-albero che contiene le chiavi $K_i \leq K < K_{i+1}$

sotto-albero che contiene le chiavi $K \geq K_F$

Albero di ricerca di ordine (grado) P

- Ogni nodo ha
  - (fino a) P figli
  - (fino a) P-1 etichette, ordinate
- Nell'i-esimo sottoalbero ci sono
  - etichette maggiori della (i-1)-esima etichetta e
  - minori della i-esima
- Albero di ricerca in memoria secondaria:
  - un nodo per blocco e quindi ogni nodo intermedio ha molti figli (un “fan-out” molto grande, pari al fattore di blocco dell’indice)
- Ogni ricerca o modifica: visita di un cammino radice-foglia
- Nel nodo, ricerca sequenziale (ma in memoria centrale!)
- La struttura è ancora (potenzialmente) rigida

B-tree

- Albero di ricerca
  - in cui ogni nodo corrisponde ad un blocco e
  - mantenuto perfettamente bilanciato (tutte le foglie sono allo stesso livello), grazie a:
    - riempimento parziale (mediamente 70%)
    - riorganizzazioni (locali) in caso di sbilanciamento

Split e merge

- Inserimenti ed eliminazioni sono precedute da una ricerca fino ad una foglia
- Per gli inserimenti,
  - se c'è posto nella foglia, ok,
  - altrimenti il nodo va suddiviso, con necessità di un puntatore in più per il nodo genitore;
    - se non c'è posto, si sale ancora, eventualmente fino alla radice. Il riempimento rimane sempre superiore al 50%
- Dualmente, le eliminazioni possono portare a riduzioni di nodi
- Modifiche del campo chiave vanno trattate come eliminazioni seguite da inserimenti

Split e merge

- Uno strumento per esercitarsi
  - https://www.cs.usfca.edu/~galles/visualization/BTree.html

Prova parziale del 28/03/2014, domanda 4
(soluzione vista a lezione con B-tree non con B+-tree)

Domanda 4 (20%)
Si consideri un B+-tree con nodi intermedi che contengono tre chiavi e quattro puntatori e foglie con tre chiavi, in cui vengano inserite chiavi (a partire dall'albero vuoto) nel seguente ordine: 13, 17, 32, 41, 57, 61, 71, 89, 21, 22. Mostrare l'albero dopo l'inserimento di cinque chiavi, di otto chiavi e alla fine. (Si ricorda che nel B+-tree ogni chiave compare in una foglia)

Mostrare poi l'albero dopo l'eliminazione della chiave 57 dall'ultimo albero ottenuto in risposta alla domanda precedente.

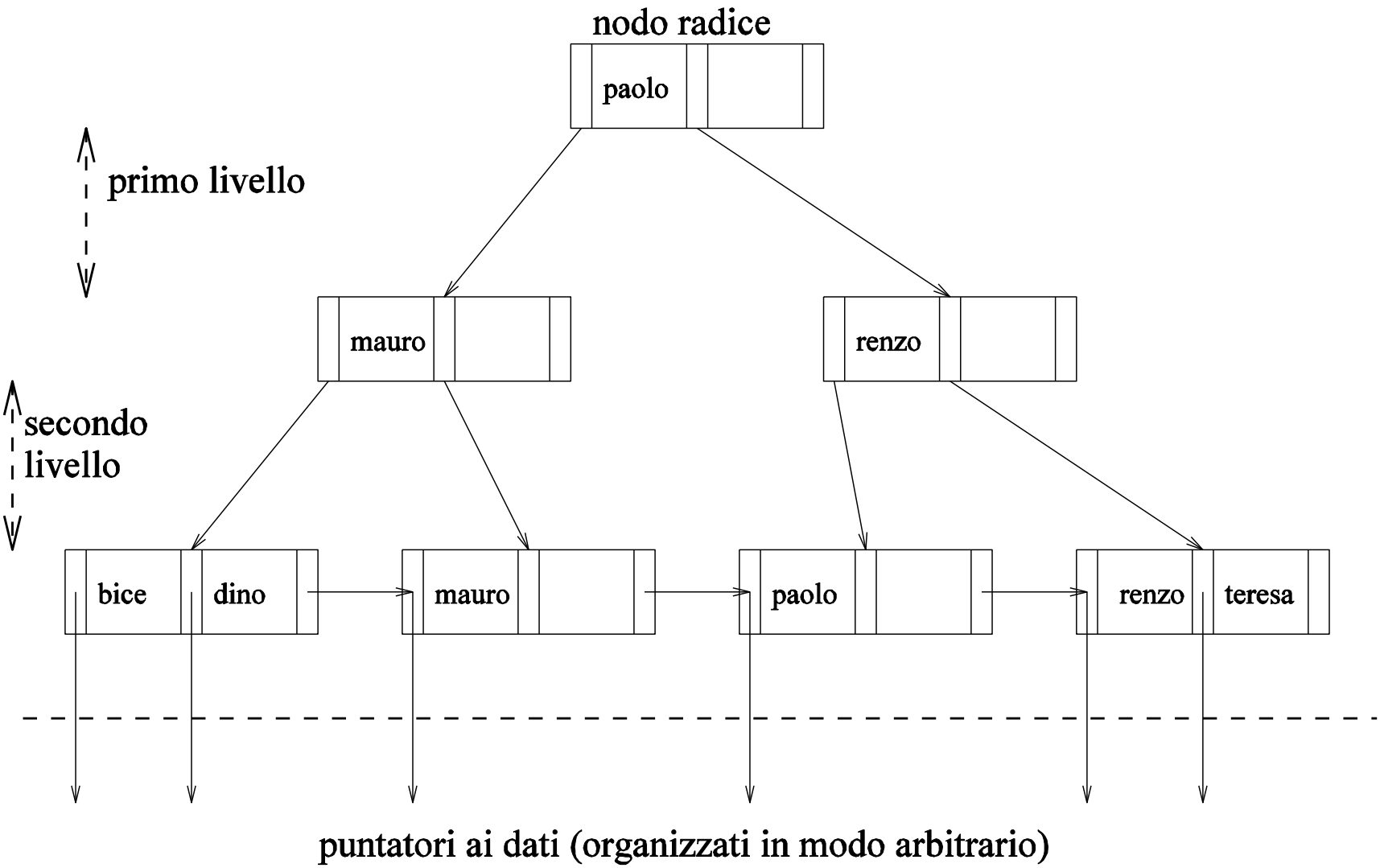
Svolto con lo strumento

- https://www.cs.usfca.edu/~galles/visualization/BTree.html

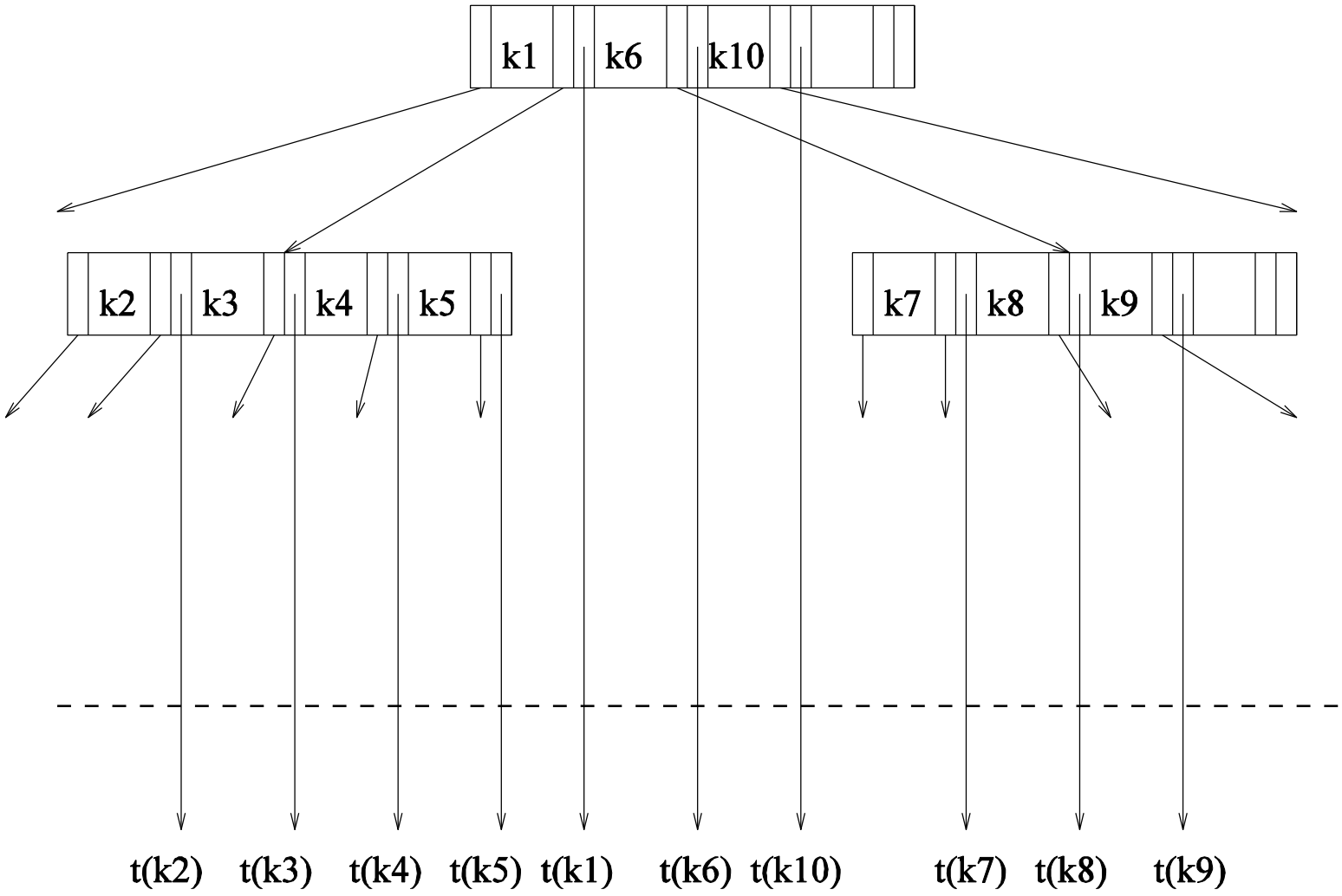
B tree e B+ tree

- B+ tree:
  - le chiavi compaiono tutte nelle foglie (e quindi quelle nei nodi intermedi sono comunque ripetute nelle foglie)
    - I riferimenti al file sono tutti nelle foglie
  - le foglie sono collegate in una lista
  - ottimi per le ricerche su intervalli
  - molto usati nei DBMS
- B tree:
  - Le chiavi che compaiono nei nodi intermedi non sono ripetute nelle foglie
    - I nodi intermedi hanno riferimenti ai blocchi del file

Un B+ tree


nodo radice
paolo
primo livello
mauro
renzo
secondo livello
bice
dino
mauro
paolo
renzo
teresa
puntatori ai dati (organizzati in modo arbitrario)

Un B-tree


k1
k6
k10
k2
k3
k4
k5
k7
k8
k9
t(k2)
t(k3)
t(k4)
t(k5)
t(k1)
t(k6)
t(k10)
t(k7)
t(k8)
t(k9)

Per esercitarsi

- Vedere differenze con esempio visto in precedenza
  - https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html
  - https://www.cs.usfca.edu/~galles/visualization/BTree.html

B tree e B+ tree, primari e secondari

- In un B+-tree
  - primario, le ennuple possono essere contenute nelle foglie
  - secondario, le foglie contengono puntatori alle ennuple
- In un B-tree
  - anche i nodi intermedi contengono ennuple (se primari) o puntatori (se secondari)

Indici, costo

- Ricerca sulla base del campo chiave:
  - Profondità dell'albero, più accesso al blocco (se serve)
    - come già detto per tutti gli indici multilivello, la profondità è di solito piccola, fra 3 e 5
  - Qualche blocco può essere nel buffer. Quanti?
    - la radice è molto probabile, se l'indice è usato
    - anche un altro livello, se viene usato molto
    - quindi il costo di un accesso diretto può scendere a 2-3 blocchi (profondità albero 4, meno 2 livelli nel buffer, più il blocco del file se interessa l'intero record)

Indici hash

- Strutture secondarie costituite da un file hash con record contenenti
  - pseudochiavi
  - puntatori ai record
- Costo della ricerca:
  - poco più di due accessi: uno (di solito, salvo overflow) all'indice e l'altro al file

Indici B-tree e indici hash, confronto

- Gli indici hash sono più efficienti per ricerche puntuali
- Però vengono usati poco
  - Sono meno "dinamici" (per esserlo serve un livello in più)
  - Non supportano ricerche su intervalli
  - I B-tree possono trarre vantaggio significativo dalla gestione dei buffer (uno o due livelli disponibili in molti casi)

Prova parziale del 28/03/2014, domanda 2

Domanda 2 (20%) Si consideri una relazione *R(CodiceImpiegato,Cognome,Nome,Mansione)* con $L = 1.000.000$ ennuple, con fattore di blocco $f_R = 100$. Con riferimento alla ricerca di tutti gli impiegati con una certa mansione, indicare il costo dell'accesso sequenziale e di quello diretto con indice su *Mansione* (con profondità $p = 4$) nei due casi seguenti (mostrare formule e valori numerici):

1. campo selettivo ($m_1 = 100.000$ valori diversi per *Mansione*)

costo accesso sequenziale:	costo accesso diretto:

2. campo poco selettivo ($m_2 = 20$ valori diversi per *Mansione*)

costo accesso sequenziale:	costo accesso diretto:

Prova parziale del 28/03/2014, domanda 2

L'esercizio richiede di confrontare l'efficienza di due strategie di recupero dati: la scansione completa della tabella e l'utilizzo di un indice secondario non clusterizzato.

1. Costo dell'Accesso Sequenziale

Il costo della scansione sequenziale dipende esclusivamente dalla dimensione fisica della relazione, indipendentemente da quanto sia selettivo il criterio di ricerca. Per calcolarlo, dividiamo il numero totale di ennuple (L) per il fattore di blocco (f_R), che indica quanti record sono contenuti in ogni blocco di memoria.

* **Formula**: L/f_R
* **Calcolo**: $1.000.000 / 100 = 10.000$ accessi.

In questo scenario, il DBMS è obbligato a leggere tutti i 10.000 blocchi che compongono il file per identificare i record che hanno la mansione cercata.

2. Costo dell'Accesso Diretto (con Indice)

Quando utilizziamo un indice, il costo totale è la somma del costo per attraversare la struttura dell'albero e del costo per recuperare i record effettivi dai blocchi della relazione.

* **Attraversamento dell'indice**: La profondità $p = 4$ indica che dobbiamo leggere 4 blocchi dell'indice per arrivare alla foglia che contiene i puntatori ai record cercati.
* **Recupero dei dati**: Poiché il campo è molto selettivo ($m_1 = 100.000$), la selettività media ci dice quanti record corrispondono a ogni valore di "Mansione". Questo si calcola come L/m_1.
* **Calcolo Selettività**: $1.000.000 / 100.000 = 10$ record.
* **Formula Totale**: $p + (L/m_1)$
* **Calcolo**: $4 + 10 = 14$ accessi.

Conclusione del Caso 1

Con soli 14 accessi rispetto ai 10.000 della scansione sequenziale, l'indice risulta estremamente efficiente perché il campo è "molto selettivo", ovvero restituisce un numero molto limitato di record rispetto alla dimensione totale della base di dati.

Ti è chiaro perché in questo caso l'indice è così vantaggioso, o preferisci analizzare subito il secondo caso con il campo "poco selettivo"?

m_2

Domanda 2 (20%) Si consideri una relazione *R(CodiceImpiegato,Cognome,Nome,Mansione)* con $L = 1.000.000$ ennuple, con fattore di blocco $f_R = 100$. Con riferimento alla ricerca di tutti gli impiegati con una certa mansione, indicare il costo dell'accesso sequenziale e di quello diretto con indice su *Mansione* (con profondità $p = 4$) nei due casi seguenti (mostrare formule e valori numerici):

1. campo selettivo ($m_1 = 100.000$ valori diversi per *Mansione*)

costo accesso sequenziale:	costo accesso diretto:
$L/f_R = 10.000$	$p + (L/m_1) = 14$

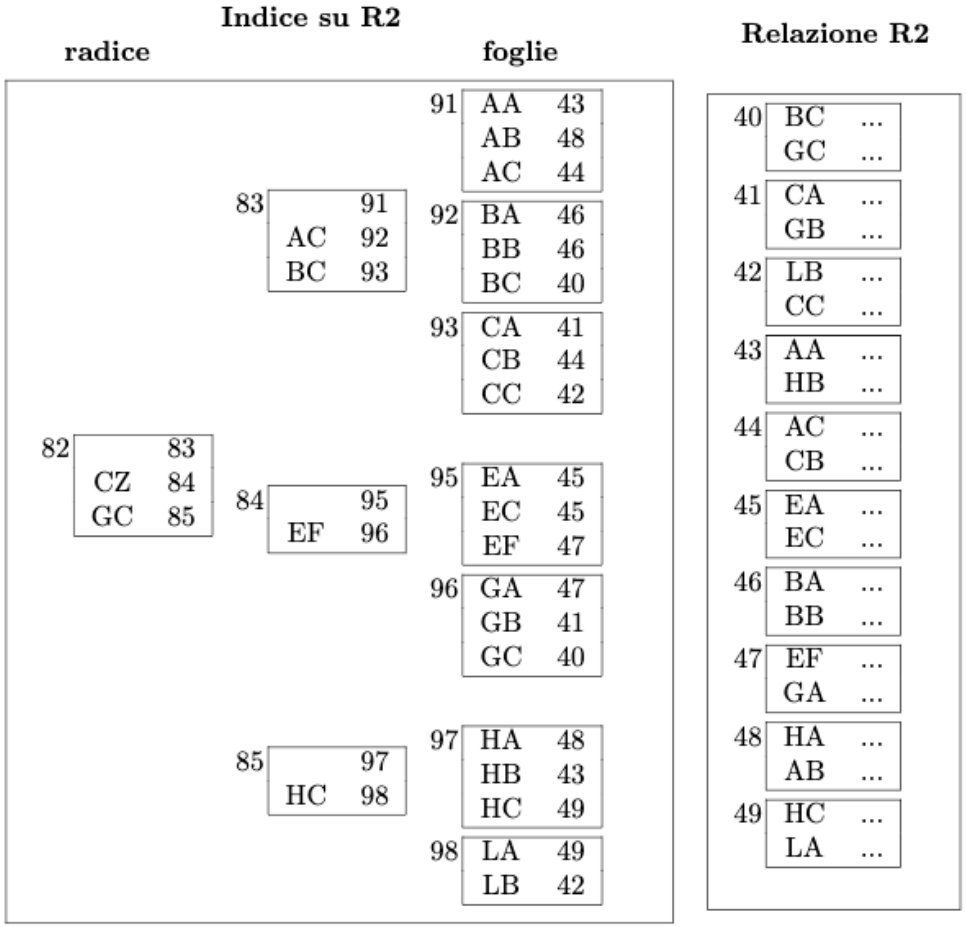
2. campo poco selettivo ($m_2 = 20$ valori diversi per *Mansione*)

costo accesso sequenziale:	costo accesso diretto:
$L/f_R = 10.000$	$p + (L/m_2) =$ ca 50.000

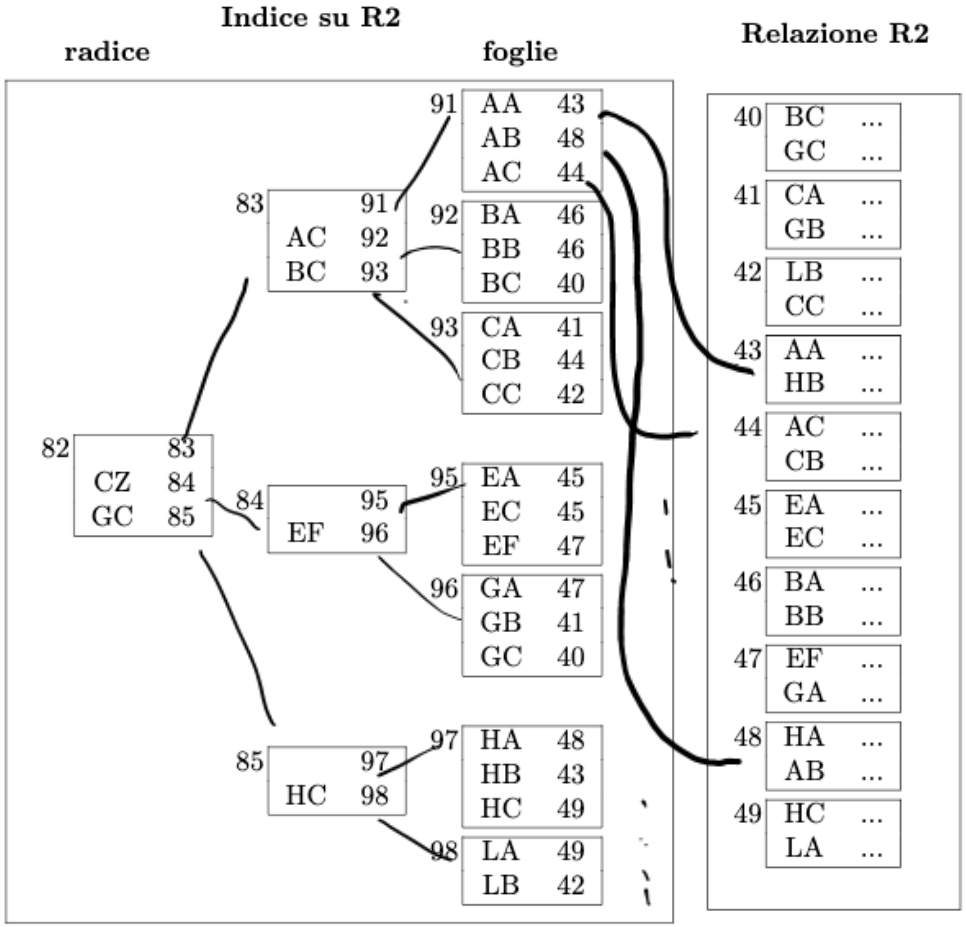
Esercizio

- Prova parziale del 28/03/2014, domanda 3 , variante

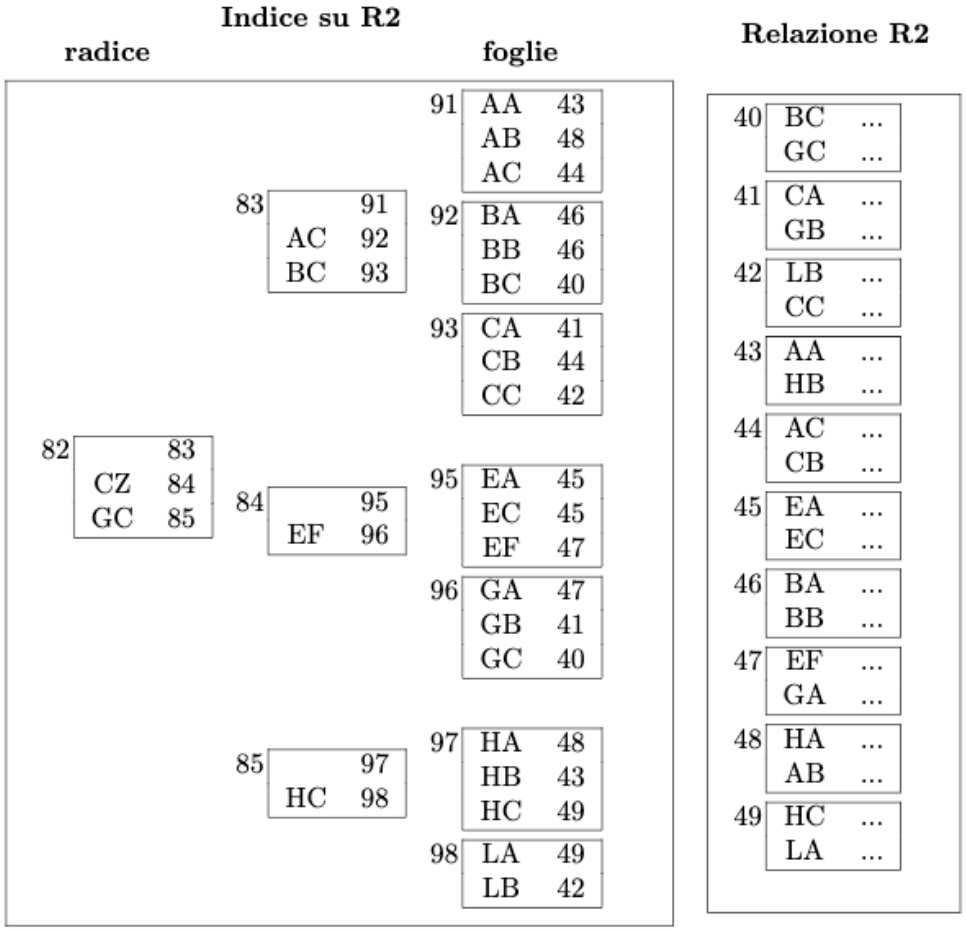
- eseguire ricerche dei record con valore della chiave AB, EA, HA, EC e indicare su quali blocchi vengono eseguite pin e quali sono fisicamente letti, con un buffer di otto pagine, politica LRU



- eseguire ricerche dei record con valore della chiave AB, EA, HA, EC e indicare su quali blocchi vengono eseguite pin e quali sono fisicamente letti, con un buffer di otto pagine, politica LRU



- eseguire ricerche dei record con valore della chiave AB, EA, HA, EC e indicare su quali blocchi vengono eseguite pin e quali sono fisicamente letti, con un buffer di otto pagine, politica LRU



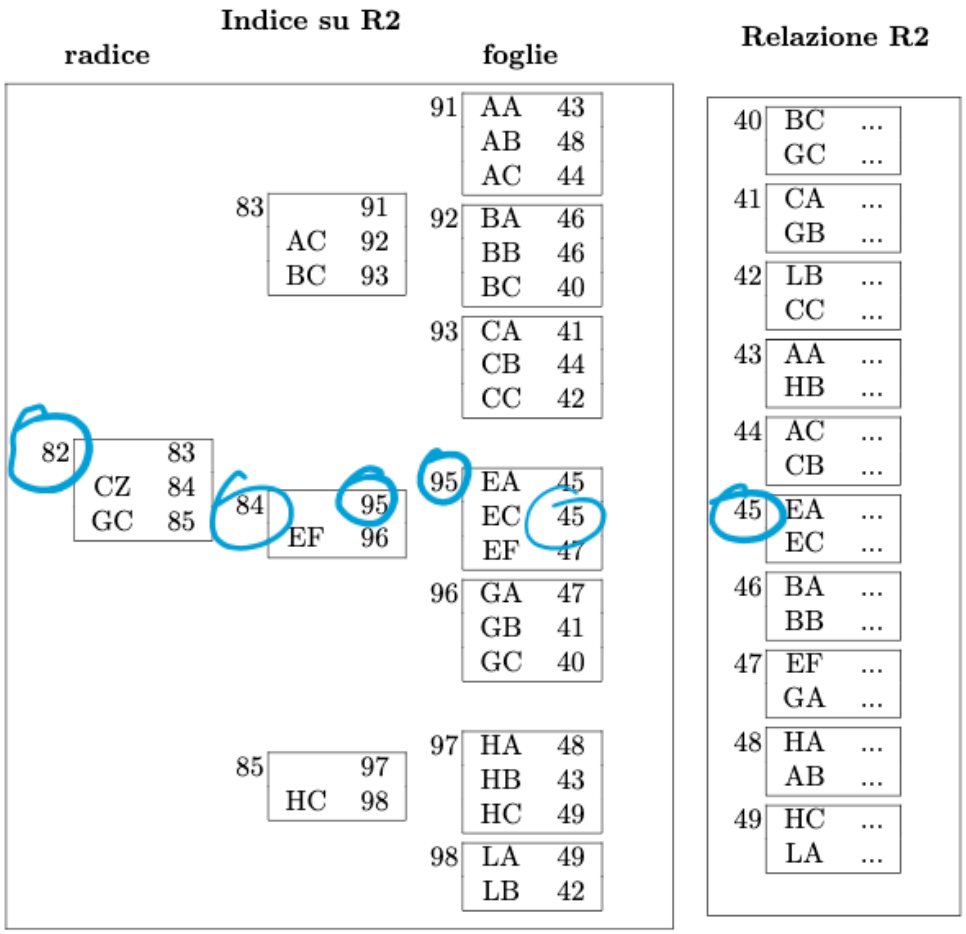
Pagina	Cont	Blocc	Unpin

Istante	Pin	Lettura

Nota

- Non mostriamo i valori del contatore, perché viene eseguito un unpin subito dopo ciascuna lettura e quindi in ogni momento c'è al più un contatore a 1 che scende a zero poco dopo

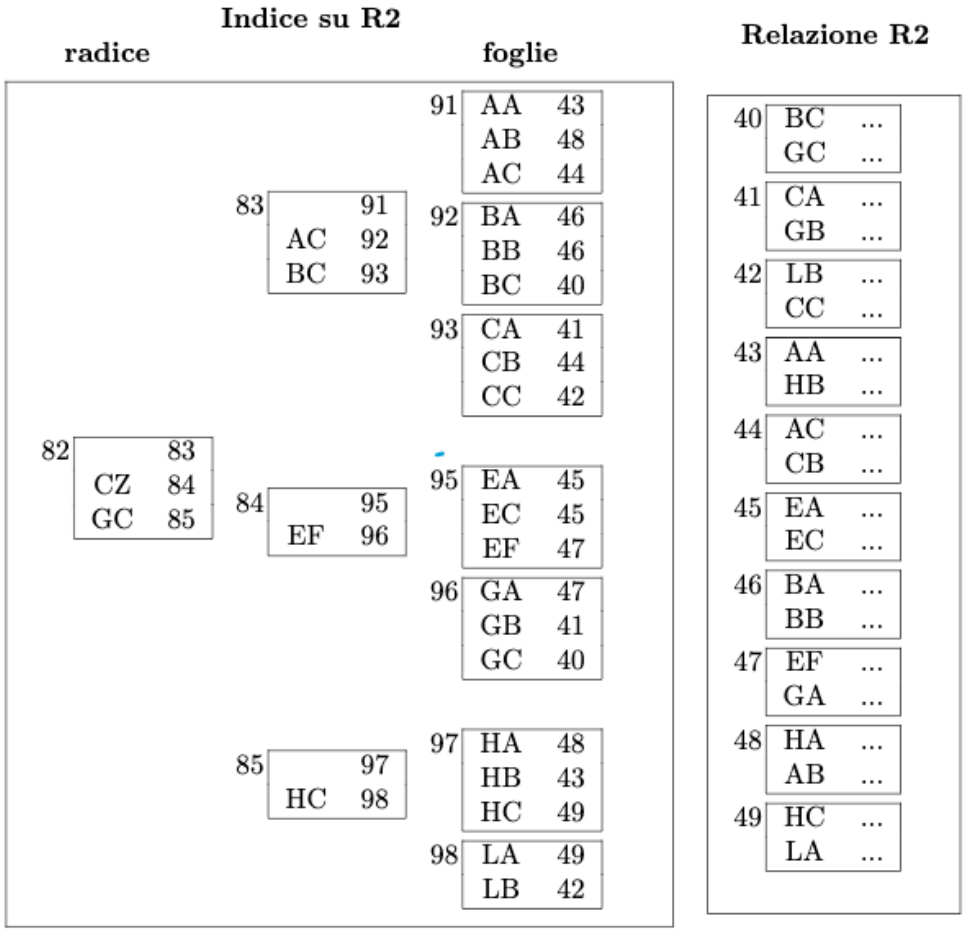
- eseguire ricerche dei record con valore della chiave AB, EA, HA, EC e indicare su quali blocchi vengono eseguite pin e quali sono fisicamente letti, con un buffer di otto pagine, politica LRU



Pagina	Cont	Blocc	Unpin
0		82	13
1		97	11
2		91	3
3		48	12
4		84	14
5		95	15
6		45	16
7		85	10

Istante	Pin	Lettura
1	82	SI
2	83	SI
3	91	SI
4	48	SI
5	82	NO
6	84	SI
7	95	SI
8	45	SI
9	82	NO
10	85	SI
11	97	SI
12	48	NO
13	82	NO
14	84	NO
15	95	NO
16	45	NO

- eseguire ricerche dei record con valore della chiave AB, EA, HA, EC e indicare su quali blocchi vengono eseguite pin e quali sono fisicamente letti, con un buffer di otto pagine, politica LRU



Pagina	Cont	Blocc	Unpin
0		82	13
1		97	11
2		91	3
3		48	12
4		84	14
5		95	15
6		45	16
7		85	10

Istante	Pin	Lettura
1	82	SI
2	83	SI
3	91	SI
4	48	SI
5	82	NO
6	84	SI
7	95	SI
8	45	SI
9	82	NO
10	85	SI
11	97	SI
12	48	NO
13	82	NO
14	84	NO
15	95	NO
16	45	NO